

Efficiency of Asynchronous Dynamic Application Security Testing: Payload Scheduling, Runtime Confirmation, and LLM-Based Triage for False-Positive Reduction

Luis Ernesto Galdámez Vides
Universidad Evangélica de El Salvador (UEES)
San Salvador, El Salvador
2026010235@cvirtualuees.edu.sv

Abstract—Dynamic Application Security Testing (DAST) scanners are widely used in continuous-integration pipelines, but their practical adoption is limited by three factors: the accumulated latency of thousands of sequential requests, the noise produced by a high false-positive rate, and the combinatorial explosion of the payload space per injection point. This paper presents and evaluates WebSecurityScanner, a DAST engine built entirely on `asyncio` that introduces (i) a payload-scheduling pipeline — context interleaving, a-priori confidence prioritization, and configurable ordering, including a live adaptive-sorting feedback loop — (ii) a runtime-confirmation stage that relabels every finding with a calibrated confidence level before it is reported, and (iii) an optional LLM-based triage stage that re-scores each raw finding using request/response context, without an additional network round trip to the target. The central hypothesis is that these techniques improve the detection-to-cost ratio (number of requests) without increasing the false-positive rate. To test it we built a reproducible testbed on OWASP Benchmark restricted to 338 GET-parameter targets (194 vulnerable instances and 144 known traps), an ablation study under a randomized complete block design (RCBD) with four request budgets and six pipeline configurations, and a battery of non-parametric tests (Friedman, Nemenyi post-hoc, paired Wilcoxon with Benjamini–Hochberg correction, and Cliff’s δ as effect size). Reproducibility is guaranteed through a per-run manifest that fixes the SHA-256 hash of the payload corpus, the git commit, and the global seed of the random-number generator. The full heuristic pipeline reaches 70.6% recall at a 1.4% false-positive rate and already saturates at 20 requests per parameter. Of the three heuristic-pipeline optimizations, only confidence-based prioritization has a practically consequential effect — and it is bounded to tight budgets: context interleaving and runtime confirmation shift the request order in a statistically detectable way (Friedman, $p \ll 0.001$) but with negligible effect sizes ($|\delta| < 0.02$). Layering LLM-based triage on top of the baseline pipeline, evaluated separately across the same budget grid, suppressed *zero* findings at every budget under its default confidence threshold, adding no measurable wall-clock overhead to

the scan; we report this null result openly and attribute it to the benchmark’s unusually unambiguous trigger signals rather than to the triage stage being validated as safe or effective in general. Run against OWASP ZAP 2.17.0 on the identical target list under the same oracle, baseline matches ZAP’s recall exactly (both detect the same 137/194 instances), at the cost of a higher false-positive rate on the known traps (ZAP: 0%). It reaches that recall $11\times$ faster than ZAP at its saturating budget. We conclude that, on a benchmark with regular trigger signals, the value of payload scheduling is one of *anticipation* (higher AUC under a tight budget), not of a higher detection ceiling. The LLM-triage stage remains to be stress-tested on a noisier target.

Index Terms—web application security, DAST, dynamic analysis, asynchronous programming, false positives, payload scheduling, LLM triage, reproducible evaluation, OWASP Benchmark.

I. INTRODUCTION

Dynamic Application Security Testing (DAST) evaluates a running system by sending manipulated requests and observing the response, with no access to source code. Compared to static analysis, its main advantage is the absence of assumptions about the target’s language or framework; its main drawback is cost. Comparative studies have repeatedly shown that black-box scanners exhibit uneven coverage and elevated false-positive rates [1], [2], [3], which erodes the development team’s trust and, in practice, leads reports to be ignored.

We identify three concrete challenges that this work addresses explicitly:

- 1) **Accumulated latency.** A realistic scan involves on the order of 10^4 – 10^5 HTTP requests. Executed sequentially, network latency dominates total run time and makes integrating DAST into every *commit* infeasible.
- 2) **Noise (false positives).** Detection based solely on a reflected string or a database error pattern produces

findings that are not exploitable. Telling an inert reflection apart from real execution requires a second, targeted observation.

- 3) **Combinatorial explosion.** The product (injection points) $\times$ (payloads per category) $\times$ (evasion transformations) grows until testing everything under a fixed time budget becomes impossible; the order in which that budget is spent determines what gets detected. This paper’s architecture is designed to withstand that growth (section II-B), but the evaluation in section IV exercises the scheduling mechanics on a constrained corpus — 338 GET parameters that saturate at 20 requests each — not a payload space large enough to demonstrate the explosion itself; section VI discusses this restriction.

A. Related work

LLM-based vulnerability triage. LLM-assisted triage of security findings is an active area outside of DAST. Static-analysis pipelines that pair a deterministic scanner (Semgrep, CodeQL) with an LLM adjudication stage report large false-positive reductions: Munson *et al.* enrich static findings with reachability context before LLM adjudication [4]; Ameen *et al.*’s multi-agent QASECCLAW raises the F_1 score of standalone Semgrep from 78.4% to 90.9% on Java test cases by suppressing non-exploitable alerts [5]; Agrawal and Ahi’s SAST-GENIUS hybrid framework cuts Semgrep’s false positives by 91% (225 to 20) through LLM-driven triage, dynamic bug descriptions, and exploit-generation validation [6]; and Du *et al.*’s industrial study at Tencent reports that hybrid LLM/static-analysis triage eliminates 94–98% of false positives while preserving recall, at a marginal cost of 2.1s to 109.5s per alert [7]. These results motivate the LLM-triage stage evaluated in section V: unlike the static-analysis setting, a DAST finding already carries live request/response evidence, so triage here re-scores an executed probe rather than a static alert. All four studies report their false-positive reductions against corpora with a substantial false-positive population to begin with; none addresses the scheduling problem that determines *which* payloads reach the target under a fixed request budget in the first place, nor evaluates triage on a target whose noise floor is already low — both of which sections IV and V take up directly.

Payload and seed scheduling. Outside DAST, the grey-box fuzzing literature treats seed/input scheduling as a first-class problem: She *et al.* prioritize fuzzing seeds by graph-centrality analysis of the program’s control-flow graph to maximize coverage growth per execution [8], the same anticipation argument this paper makes for confidence-based payload prioritization (sections II-B and IV-G) — earlier detection per request under a fixed budget, not necessarily a higher final yield. To our knowledge no prior work transfers this scheduling framing to black-box DAST payload selection, where the search space is a fixed, tester-supplied corpus rather than a mutation-generated one.

OWASP Benchmark as an evaluation target. Potti *et al.* benchmark OWASP ZAP v2.12.0 and v2.13.0 on the OWASP Benchmark project, reporting an aggregate true-positive rate of 68.75–74.63% at a 0.86–3.45% false-positive rate across vulnerability categories [9]. Section IV-F runs a newer ZAP release (2.17.0) ourselves against this paper’s own 338-target list under the same oracle used throughout this paper, and finds their published range consistent with our own single-run result.

Contributions. This paper makes the following contributions:

- An asynchronous DAST engine with a single HTTP choke point, back-pressured concurrency control, token-bucket rate limiting, and safeguards against SSRF in redirect following (section II).
- A payload-scheduling pipeline with decoupled stages — context interleaving, prioritization, ordering, and a live adaptive-sorting feedback loop — plus a runtime-confirmation stage that calibrates finding confidence (sections II and III).
- An optional LLM-based triage stage that re-scores executed probes using request/response context, and an honest evaluation of it against the same budget grid, including a null-result case where it suppresses nothing and a measured wall-clock overhead of zero (section V).
- A non-blocking `TelemetryWorker` that logs one JSONL row per payload probe, enabling instance-by-instance detection-cost measurement (section II-D).
- A reproducible evaluation methodology: a containerized testbed, an independent grading oracle, an RCBd experimental design, and a reproducibility manifest with a corpus hash (section III).
- An ablation study that isolates each pipeline stage’s contribution to the detection-versus-cost curve and to the false-positive rate (section IV).
- An external baseline against OWASP ZAP on the identical target list under the same oracle, reported honestly including the categories where ZAP outperforms WEBSecurityScanner (section IV-F).

The rest of the paper is organized as follows. Section II describes the engine’s design. Section III details the testbed and the experimental design. Section IV presents the ablation evaluation and section V the LLM-triage evaluation. Section VI discusses threats to validity, and section VII concludes.

II. ARCHITECTURE AND DESIGN

WEBSecurityScanner is structured into four subsystems: the asynchronous HTTP core, the payload loader, the vulnerability-tester hierarchy with its scheduling pipeline, and the telemetry/reproducibility instrumentation. Figure 1 summarizes the workflow.

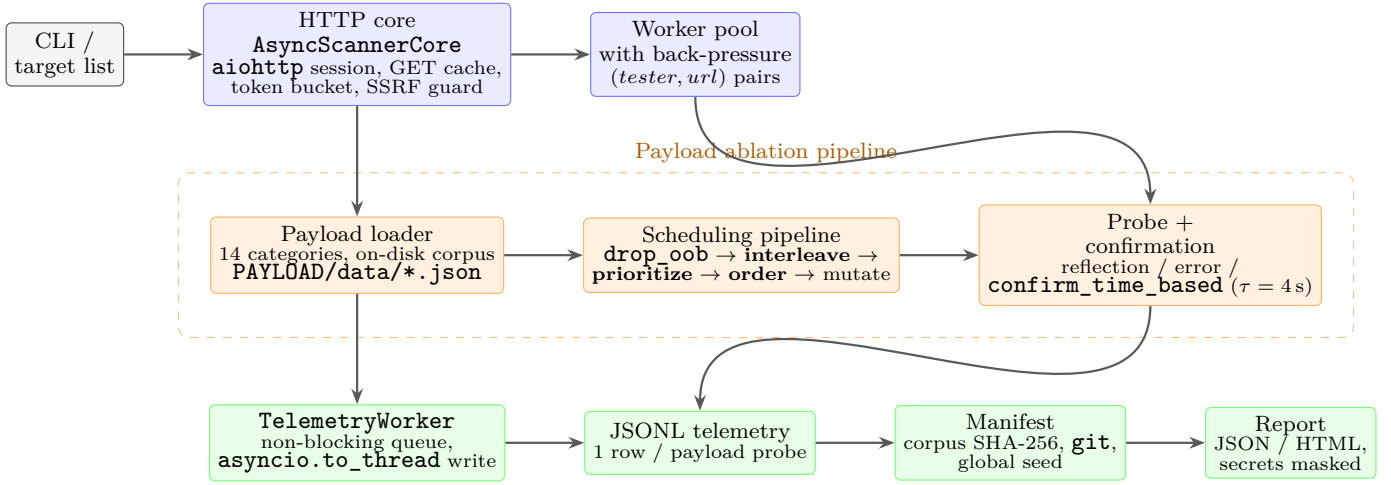


Figure 1. Workflow of WEBSECURITYSCANNER. The asynchronous HTTP core concentrates all network I/O; the worker pool dispatches $(tester, url)$ pairs with back-pressure. Each tester loads its payload corpus and runs it through the scheduling pipeline before probing the target. The confirmation stage relabels finding confidence. **TelemetryWorker** writes one JSONL row per probe without blocking the event loop; the manifest fixes the reproducibility inputs.

A. Asynchronous HTTP core

All network traffic goes through `AsyncScannerCore.request()`, a single choke point built on an `aiohttp.ClientSession`. Centralizing I/O lets the engine uniformly apply: (a) a response cache for 200-status GET requests; (b) token-bucket rate limiting; (c) seedable User-Agent rotation; (d) separate timeouts for connection establishment and socket reads; (e) a 5 MiB cap on the response body, with chunked reading and a truncation flag; and (f) a Server-Side Request Forgery safeguard that rejects redirects toward private addresses unless explicitly authorized. The `elapsed` field the core returns is monotonic and spans redirects; for per-probe latency measured for scientific purposes, `time.perf_counter()` is instead measured at the tester boundary.

Work dispatch uses `run_worker_pool()`, which enforces back-pressure through a bounded queue (`queue_factor`) so that target discovery cannot overrun memory or saturate the target. **SIGINT** handling is orderly: the queue is drained and the session is closed idempotently.

B. Payload-scheduling pipeline

Every tester derives from a common base class that resolves payload loading through `load_payloads()`. Internally, this routine applies, in order, the following stages, all of them independently toggleable for the ablation study:

- 1) `_drop_useless_oob`: discards out-of-band payloads when no collector is configured.
- 2) `_interleave_by_context` (INTERLEAVE): reorders the list to alternate across syntactic contexts (HTML, attribute, JavaScript, URL, SQL, ...) instead of exhausting one context before moving to the next. The intuition is that, under a tight budget, covering one representative payload per context first maximizes the probability of an early hit.

- 3) `_prioritize` (PRIORITIZE): a stable sort by (a-priori confidence, severity), so that vectors with higher historical success probability and greater impact are tried first.
- 4) `_apply_payload_order` (ORDER): a configurable terminal policy — **natural** (identity), **random** (seeded shuffle), or **freq** (most-populated context groups first).
- 5) **Live adaptive sorting** (ADAPTIVE): a feedback loop that re-ranks the remaining queue mid-scan using the hit/miss outcomes observed so far for each context, in place of the a-priori **freq** order fixed before the scan starts. Disabling it (`-no-adaptive-sorting`) pins the pipeline to the static a-priori order and isolates the contribution of the online feedback loop (section IV).
- 6) `_apply_runtime_mutations`: WAF-evasion transformations and case variation applied at send time.

C. Runtime confirmation and calibrated confidence

Every finding carries a confidence level from the ordered domain `{LOW, MEDIUM, HIGH, CONFIRMED}`. The corpus assigns an a-priori confidence to each payload; the confirmation stage produces the *final* confidence:

- **Time-based SQL injection.** Baseline latency is resampled (`measure_baseline_latency`) and the response to the timed payload is contrasted against a threshold $\tau = 4\text{ s}$ (`confirm_time_based`); only if the difference exceeds the threshold is the finding marked **CONFIRMED**.
- **XSS.** A raw reflection is refined according to the HTML context in which it appears (`_assess`), promoting it to **HIGH** or downgrading it to **LOW**.
- Under the `-no-runtime-confirm` ablation, both branches are short-circuited and the finding is reported with the payload’s a-priori confidence,

leaving a record (`verification=skipped`) in the evidence. The probe is logged to telemetry either way.

D. Non-blocking telemetry

`TelemetryWorker` decouples event generation from persistence. Its `record()` method is synchronous and non-blocking: it enqueues the row and assigns a `request_index` (run-scoped, monotonic), an ISO-8601 UTC timestamp, and a `run_id`. An asynchronous consumer batches rows and writes them through `asyncio.to_thread()`, so the event loop never blocks on disk I/O. If the queue fills up, excess rows are dropped and counted (`dropped`), preserving the back-pressure property.

The **granularity** is deliberate: one JSONL row per *payload probe*, after the tester’s confirmation heuristics, not per socket write. Baseline and confirmation sub-requests are not logged individually. Each row’s schema includes `run_id`, `timestamp`, `tester_id`, `payload_id`, `context`, `confidence_apriori`, `url`, `method`, `elapsed_time`, `decision`, `confidence_final`, and `request_index`. This trace is the primary input for the detection-cost analysis in section IV.

E. Reproducibility: the per-run manifest

Alongside telemetry, every instrumented run emits a JSON manifest that fixes: the schema version, the `run_id`, the creation timestamp, the `git` commit (`unknown` when not applicable), the RNG’s global seed, and a `corpus` block with the SHA-256 hash computed over the sorted `PAYLOAD/data/*.json` files (each file name is mixed into the digest) plus the file list and its count. The complete run configuration dictionary is also included. Hash computation and the `git` query run on a separate thread; a failure is logged and ignored, without aborting or delaying the scan. This artifact makes it possible to verify *a posteriori* that two runs share exactly the same corpus and configuration.

III. EXPERIMENTAL METHODOLOGY

A. Testbed

The evaluation target is OWASP Benchmark [10], a deliberately vulnerable Java application with labeled test cases. It is deployed via `docker compose`, serving HTTPS on port 8443 with a self-signed certificate (hence every probe uses `--no-verify-ssl`). To bound the study to a single vector type, the benchmark is restricted to the **338 GET-parameter cases** (`vector = getparam`). The `build_ground_truth.py` generator produces the ground-truth file as an array of `<url,param,type,vulnerable>` records; records with `vulnerable = false` are explicit *traps* (known-safe points that must not fire). Table I summarizes its composition.

B. Grading oracle

Grading is performed by `tools/eval_oracle.py`, an independent script with no third-party dependencies. It normalizes every `(url,param,type)` tuple with the same

Table I
GROUND-TRUTH COMPOSITION (OWASP BENCHMARK, `GETPARAM` VECTORS).

Category	Vulnerable instances	Traps
Reflected XSS	68	
SQL Injection	65	
Path Traversal	30	144
Command Injection	25	
LDAP Injection	6	
Total	194	144

folding functions the scan pipeline uses (query string and fragment are dropped; type aliases collapse onto the same bucket), and computes:

$$\begin{aligned} TP &= |H \cap P|, & FP &= |H \setminus P|, \\ FN &= |P \setminus H|, & FPR &= \frac{|H \cap T|}{|T|}, \end{aligned} \quad (1)$$

where H is the set of findings, P the set of vulnerable instances, and T the set of traps. FPR is defined *exclusively* over the known-trap population; when ground truth defines no traps, N/A is reported. Precision, recall, and F_1 are derived from these.

C. Experimental design

A **randomized complete block design** (RCBD) is used, where:

- The **factor** is the pipeline configuration, with five heuristic-pipeline levels — **baseline** (all optimizations on, including the live adaptive-sorting loop), **no-interleave**, **no-priority**, **no-runtime-confirm**, and **no-adaptive-sorting** — i.e., a single-component ablation per arm. A sixth arm, **ai-triage**, layers the LLM-based triage stage on top of **baseline** and is evaluated and reported separately (section V), since it targets false-positive suppression rather than request scheduling.
- The **request budget** (`--max-payloads` $\in \{10, 20, 50, \infty\}$) is treated as a blocking factor: detection is trivially monotone in the budget, and blocking on it isolates the configuration effect.
- The **blocking unit** for significance testing is the *per-vulnerable-instance detection cost*: the `request_index` of the first probe that detects it, censored at (budget + 1) when never detected.

The full ablation grid is $4 \times 5 = 20$ runs, orchestrated by `tools/run_experiments.py` and consolidated into `testbed/experiment_results.csv`; all 20 were graded successfully (section IV). The **ai-triage** arm adds 4 further runs (one per budget) on top of **baseline**, graded and reported in section V. Every run produces its own JSONL telemetry, its JSON report, and its `metrics.json`.

D. Efficiency metric: detection-curve AUC

For each run we build the step curve $c(k) = |\{\text{instances detected with } \texttt{request_index} \leq k\}|$. Taken on normalized axes (recall versus fraction of budget), its area

$$\text{AUC} = \int_0^1 \frac{c(k)}{|P|} d\hat{k} \quad (2)$$

equals mean recall over the budget and is the primary efficiency measure: a pipeline that detects the same set but earlier has a higher AUC.

E. Statistical tests

For every budget at which **baseline**, **no-interleave**, **no-priority**, and **no-runtime-confirm** completed, the four detection-cost vectors are compared with:

- 1) a **Friedman** test [11] across the configurations (blocks = vulnerable instances);
- 2) a **Nemenyi** [12] post-hoc for pairwise comparisons;
- 3) **Wilcoxon** signed-rank and **McNemar** tests for **baseline** versus each ablation, with **Benjamini–Hochberg** [13] correction across the family of pairs;
- 4) **Cliff’s δ** [14] as a non-parametric effect size (a $\delta > 0$ indicates the ablation is slower or worse).

This protocol follows Demšar’s recommendations for comparing methods across multiple datasets [15]. Blocks are further pooled across all budgets for a global test. The fifth arm, **no-adaptive-sorting**, is contrasted separately against **baseline** as a paired adaptive-versus-static test on first-true-positive cost (section IV-E), since it targets the live feedback loop rather than a static scheduling stage.

F. Reproducibility pillars

Every reported run ships with: (i) the manifest of section II-E, carrying the corpus’s SHA-256 hash and the **git** commit; (ii) a global seed (**--global-seed**) that fixes the **random** payload ordering, User-Agent rotation, jitter, and case mutation; and (iii) the orchestration script, which is idempotent — an existing results directory is skipped unless **--force** is given, so an interrupted sweep resumes without recomputation.

IV. EVALUATION AND RESULTS

The ablation sweep produced 20 graded runs: the four budgets $\{10, 20, 50, \infty\}$ with all 5 heuristic-pipeline configurations (4 complete blocks). All values come from **tools/analyze_results.py**.

A. Efficiency: detection versus budget

Figure 2 shows that **baseline** recall grows from 0.675 at 10 requests to its saturation value 70.6% already at 20 requests per parameter, and does not improve when the budget is widened to 50 or removed entirely. The **no-interleave**, **no-runtime-confirm**, and **no-adaptive-sorting** curves are indistinguishable from **baseline**’s; only **no-priority** separates, and only at the intermediate budget of 20 (fig. 3).

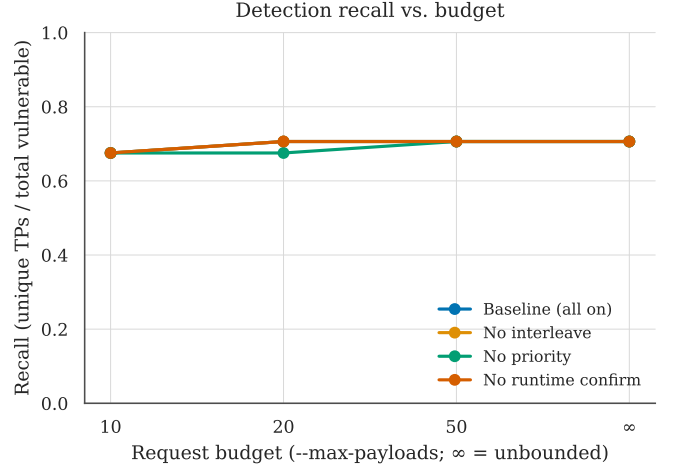


Figure 2. Recall (unique vulnerable instances detected / total) as a function of the **--max-payloads** request budget (∞ = unbounded), one curve per pipeline configuration. The **baseline**, **no-interleave**, **no-runtime-confirm**, and **no-adaptive-sorting** curves overlap.

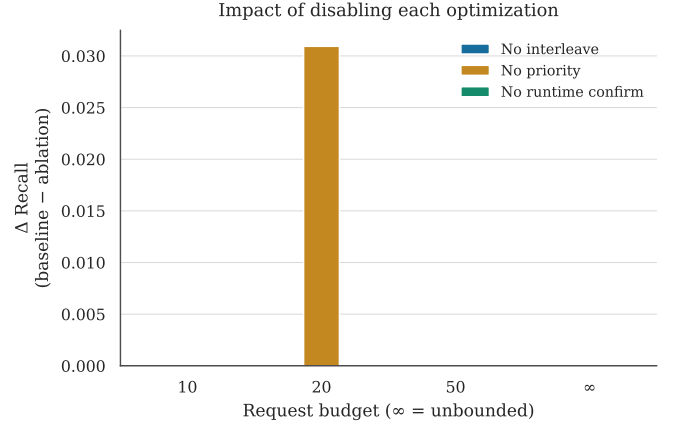


Figure 3. Recall drop (baseline – ablation) from disabling each optimization, by budget. The only sizeable effect is **no-priority** at the 20-request budget.

B. Metric summary and AUC

At a budget of 20 requests per parameter, **baseline** reaches a detection-curve AUC of 0.356 (eq. (2)), with precision 64.0%, recall 70.6%, and F_1 67.2%, at a 1.4% false-positive rate (2 of 144 traps). Disabling context interleaving, runtime confirmation, or adaptive sorting leaves recall and AUC unchanged (0.356, 0.355, and 0.355 respectively), but each trips one additional known trap (FPR 2.1%, 3 of 144), fractionally lowering precision to 63.7% and F_1 to 67.0% — a difference of a single false positive out of 210 findings. Disabling prioritization is the only change with a practically visible cost: AUC drops to 0.350, recall falls by 3.1 p.p., and precision by 5.5 p.p., which in absolute terms amounts to 6 undetected vulnerabilities and 16 additional false positives relative to **baseline**. At budgets 50 and ∞

Table II
PER-RUN METRICS AND DETECTION-CURVE AUC (EQ. (2)). SOURCE:
SUMMARY_BY_RUN.CSV.

Budget	Config.	Prec.	Rec.	F_1	AUC
10	baseline	0.642	0.675	0.658	0.347
	no-interleave	0.642	0.675	0.658	0.347
	no-priority	0.642	0.675	0.658	0.348
	no-runtime-confirm	0.649	0.675	0.662	0.351
	no-adaptive-sorting	0.642	0.675	0.658	0.347
20	baseline	0.640	0.706	0.672	0.356
	no-interleave	0.637	0.706	0.670	0.356
	no-priority	0.585	0.675	0.627	0.350
	no-runtime-confirm	0.637	0.706	0.670	0.355
	no-adaptive-sorting	0.637	0.706	0.670	0.355
50	baseline	0.591	0.706	0.643	0.359
	no-interleave	0.591	0.706	0.643	0.366
	no-priority	0.591	0.706	0.643	0.360
	no-runtime-confirm	0.591	0.706	0.643	0.359
	no-adaptive-sorting	0.591	0.706	0.643	0.366
∞	baseline	0.591	0.706	0.643	0.369
	no-interleave	0.591	0.706	0.643	0.369
	no-priority	0.591	0.706	0.643	0.369
	no-runtime-confirm	0.591	0.706	0.643	0.369
	no-adaptive-sorting	0.591	0.706	0.643	0.369

Table III
TESTS ON PER-INSTANCE DETECTION COST, **BASELINE** VERSUS EACH
ABLATION (POOLED BLOCK, $n = 776$). WILCOXON WITH
BENJAMINI-HOCHBERG CORRECTION; $p_{\text{Nem.}}$ FROM THE NEMENYI
POST-HOC; CLIFF'S δ AS EFFECT SIZE. SOURCE: STATS.JSON.

baseline vs	Mean cost	p_{BH}	$p_{\text{Nem.}}$	Cliff's δ	Magnitude
no-interleave	40 247 $\rightarrow$ 40 042	9.1×10^{-31}	< 0.001	-0.001	negligible
no-priority	40 247 $\rightarrow$ 39 847	4.4×10^{-103}	< 0.001	-0.012	negligible
no-runtime-confirm	40 247 $\rightarrow$ 40 226	3.6×10^{-4}	0.361	-0.001	negligible

Global Friedman: $\chi^2 = 1001.7$, $p = 7.8 \times 10^{-217}$.

the effect dissolves entirely: all five configurations converge to *identical* precision, recall (70.6%), and F_1 (table II).

C. Statistical significance

Over the per-instance detection-cost vectors (pooled block across the 4 complete budgets, $n = 776$), the Friedman test rejects the equivalence of the four configurations ($\chi^2 = 1001.7$, $p = 7.8 \times 10^{-217}$). However, the effect size is *negligible* in the pairwise **baseline**-versus-ablation comparisons: Cliff's δ of -0.001 (**no-interleave**), -0.012 (**no-priority**), and -0.001 (**no-runtime-confirm**), all with $|\delta| < 0.02$ (table III). The fifth arm, **no-adaptive-sorting**, is analyzed separately in section IV-E. The Nemenyi post-hoc fails to separate **no-runtime-confirm** from **baseline** ($p = 0.361$). The reading is that the differences Friedman detects are of *fine-grained request ordering* — statistically real given the large number of blocks, but with no practical translation into the final detected set. The only operationally consequential change is the loss of selectivity of **no-priority** under tight budgets, already described in section IV-B; note that **no-priority** even lowers the mean detection cost (40 247 $\rightarrow$ 39 847 requests), i.e., it hits earlier on average but at the cost of precision.

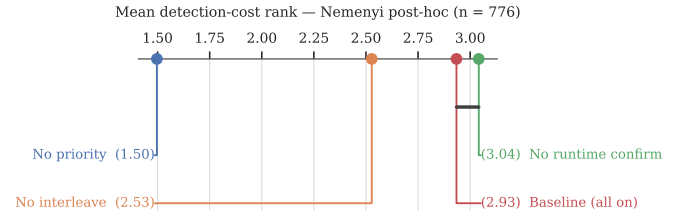


Figure 4. Critical-difference diagram (Nemenyi post-hoc, $\alpha = 0.05$) on mean rank of per-instance detection cost. Lower rank = earlier detection. Configurations joined by a horizontal bar are not distinguishable. Since all effect sizes are negligible (table III), the diagram is indicative of ordering, not of a practical advantage.

D. Critical-difference diagram

Figure 4 ranks the four Friedman-tested configurations by mean detection-cost rank (pooled block, $n = 776$). The Nemenyi post-hoc leaves only **baseline** and **no-runtime-confirm** unseparated (horizontal bar); **no-interleave** sits at the worst rank (latest detection) and **no-priority** at the best. This last point must be read carefully: **no-priority** advances the mean first detection because it tries apparently high-yield trivial payloads first, but —as table II shows— it does so at the cost of 6 vulnerabilities and 16 false positives. The diagram compares *when* each configuration detects, not *how much* nor at what precision.

E. Adaptive sorting: live feedback versus static a-priori order

Figure 5 contrasts the per-instance first-true-positive request cost between **baseline** (live adaptive sorting) and **no-adaptive-sorting** (static a-priori order), pooled across budgets. The paired Wilcoxon test and Cliff's δ (section III-E) quantify whether re-ranking the queue mid-scan from observed hit/miss feedback shifts first-detection cost relative to committing to the **freq** order fixed before the scan starts.

F. External baseline: OWASP ZAP on the same target list

To ground **baseline** against an external scanner rather than only against its own disabled stages, we ran OWASP ZAP 2.17.0 (default active-scan policy, default attack strength and alert threshold) against the identical 338-target list, one URL-scoped active scan per target (**recurse=false**), graded by the same independent oracle against the same ground truth. Table IV reports the result, both over every alert ZAP raised (*unrestricted*) and restricted to the five vulnerability categories this ground truth actually scores — **SQLInjection**, **XSS**, **PathTraversal**, **CommandInjection**, and **LDAPInjection** — since ZAP's default policy also reports categories (missing security headers, cookie flags, CSP configuration, ...) that **WEBSECURITYSCANNER**'s testers do not attempt and that consequently have no population to be scored against in this ground truth.

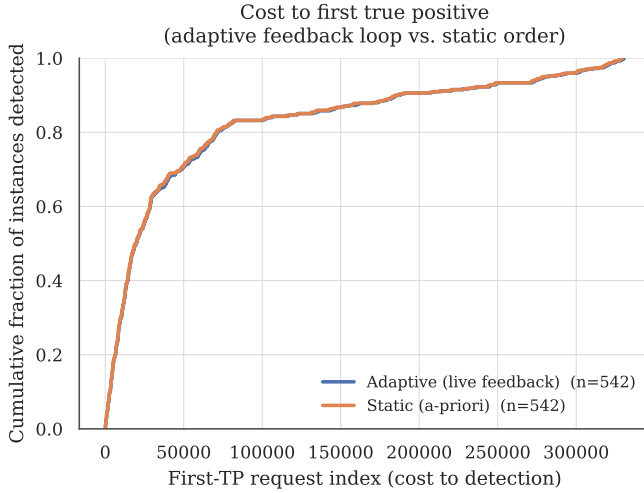


Figure 5. ECDF of the per-instance first-true-positive request index: adaptive (**baseline**) versus static a-priori order (**no-adaptive-sorting**), pooled across budgets.

Table IV

OWASP ZAP 2.17.0 VERSUS WEBSecurityScanner’s **BAS**ELINE, SAME 338-TARGET LIST, SAME ORACLE. ZAP’s **RESTRICTED** COLUMN DROPS ALERT CATEGORIES OUTSIDE THIS GROUND TRUTH’S FIVE SCORED TYPES. SOURCE: `TOOLS/RUN_ZAP_BAS`ELINE.PY.

	Prec.	Rec.	F_1	FPR	Wall-clock
ZAP (unrestricted)	0.252	0.706	0.371	0.000	1961 s
ZAP (restricted)	0.890	0.706	0.787	0.000	1961 s
WEBSecurityScanner (budget 20)	0.640	0.706	0.672	0.014	183 s
WEBSecurityScanner (budget ∞)	0.591	0.706	0.643	0.021	1609 s

Three results are worth stating plainly, including the ones that do not favor WEBSecurityScanner. First, **recall is identical**: ZAP and **baseline** both detect exactly 137 of the 194 vulnerable instances, at every budget — the same subset, since both tools’ misses are concentrated in the same harder categories. This is independent evidence for this paper’s central claim about the target, not the tool: OWASP Benchmark’s 57 undetected instances resist a second, independently engineered scanner just as they resist this one, which is far more consistent with a property of the benchmark than of either tool. Second, **ZAP’s FPR on this run is 0%** (zero trap hits out of 144) versus **baseline**’s 1.4%–2.1%: on the specific 144 known-safe traps in this ground truth, ZAP’s default policy is strictly more conservative than WEBSecurityScanner’s heuristic confirmation stage. Third, **restricted precision favors ZAP**, but the comparison is apples-to-oranges by construction: ZAP’s default policy is a general-purpose scanner run unfiltered, while **baseline** tests only the five categories this ground truth scores, so *restricted* precision — computed after manually filtering ZAP down to the same five categories — necessarily flatters ZAP relative to its own unfiltered, real-world operating point. Read this row as a best-case number for ZAP, not a fair head-to-head:

restricted precision is 89.0% versus 64.0% for **baseline** at budget 20, while ZAP’s actual out-of-the-box (*unrestricted*) precision is 25.2%, since its default policy reports 582 raw alerts across every category it tests, only 154 of which fall inside this ground truth’s five scored types. The wall-clock gap is large in the other direction: ZAP’s default active scan takes 1961s for these 338 targets, comparable to WEBSecurityScanner’s *unbounded*-budget run rather than its saturating 20-request budget, which reaches the same recall in 183s — roughly $11\times$ faster at the budget where **baseline** already saturates (section IV-A).

This is a single run of each tool, not a repeated-trials comparison, and the two scanners’ default policies test different rule sets by design (ZAP’s is far broader), so table IV should be read as one honest data point, not a leaderboard result. Potti *et al.* independently report OWASP ZAP v2.12.0/v2.13.0 aggregate results on the full OWASP Benchmark project (all vector types, not just GET) of 68.75–74.63% recall at 0.86–3.45% FPR [9]; our single-run, GET-only, ZAP-2.17.0 result of 70.6% recall at 0% FPR is consistent with that range.

G. Summary of findings

- 1) The **baseline** pipeline saturates at 70.6% recall and 1.4% FPR with only 20 requests per parameter; widening the budget further adds no detection, only false positives (precision drops from 64.0% to 59.1% between the 20 and ∞ budgets).
- 2) **Confidence-based prioritization** is the only stage whose removal has a measurable cost, and it concentrates at tight budgets: -3.1 *p.p.* recall and -5.5 *p.p.* precision at budget 20, an effect that disappears from 50 onward.
- 3) **Context interleaving, runtime confirmation, and live adaptive sorting** do not change recall, precision, or FPR on this benchmark; their effect on request order is statistically detectable but negligible in magnitude.
- 4) Consequently, the value of payload scheduling in this scenario is one of *anticipation* (higher AUC under a tight budget), not of a higher detection *ceiling*.

V. LLM-BASED TRIAGE EVALUATION

Beyond the scheduling pipeline, WEBSecurityScanner offers an opt-in LLM-based triage stage (`--enable-ai-triaging`) that re-scores every raw finding using the request/response evidence already collected by the heuristic engine, without an additional network round trip to the target. Each candidate finding is sent to an LLM backend (`AgentClient`) together with its payload, response snippet, and heuristic confidence; the model returns a suppression probability, and findings scoring above `ai_fp_threshold` ($= 0.75$ by default) are dropped before the report is written. Degradation is transparent: if `ai_module` is unavailable, the backend fails its health check,

Table V
LLM TRIAGE AUDIT TRAIL, **AI-TRIA**GE ARM (**BASELINE** PIPELINE +
LLM-BASED TRIAGE, **AI_FP_THRESHOLD**= 0.75). SOURCE:
EXPERIMENT_RESULTS.CSV.

Budget	Triaged	Suppr.	FP suppr.	FN introd.
10	441	0	0	0
20	459	0	0	0
50	476	0	0	0
∞	476	0	0	0

or a per-call request errors, the tester falls back to the deterministic heuristic decision and the scan continues.

A. Setup

The **ai-triage** arm layers this stage on top of **baseline** — identical scheduling pipeline, identical corpus, identical budgets — so any difference against **baseline** isolates the triage stage’s effect. Table V reports, per budget: the number of candidates the model triaged (**AI_Triaged**), how many it suppressed (**AI_Suppressed**), how many of those were true false positives (**AI_FP_Suppressed**), and how many were real vulnerabilities the model incorrectly hid (**AI_FN_Introduced**).

B. Latency overhead

Triage calls run through a **BatchingTriageClient** that queues candidates and issues them off the event loop, overlapped with the scan’s own in-flight requests, rather than blocking on a round trip per finding. Comparing the wall-clock span of each **ai-triage** run against its **baseline** counterpart at the same budget (identical telemetry row count, so identical work) isolates the added cost: 12.2s at budget 10, −32.2s at budget 20, 0.6s at budget 50, and −41.3s at budget ∞ (a 1568s run) — within ordinary run-to-run variance in both directions, not a systematic overhead. This does not mean LLM inference is free: it means the async batching design keeps its latency off the scan’s critical path for this backend and candidate volume (1852 candidates across four runs). A backend with materially higher per-call latency, or a target with a much larger candidate volume, could still saturate the batching queue; that stress case is not exercised here.

C. Result: null suppression under the default threshold

Across every evaluated budget, the LLM triage stage suppressed **zero** findings: 1852 candidates were sent to the model and none scored above the 0.75 suppression threshold. As a direct consequence, **ai-triage** precision, recall, and F_1 are identical to **baseline**’s at every budget (table II versus table V), and no false negative was introduced.

This is a legitimate negative result, not an inconclusive one, and it is consistent with the same root cause identified in the scheduling ablation (section IV-G): OWASP Benchmark’s trigger signals are unusually regular. Its reflected-XSS and error-based-SQLi findings carry unambiguous

textual evidence (the payload verbatim in the response, or a database-specific error string), which is exactly the kind of signal a threshold-0.75 triage model is calibrated *not* to override, since doing so risks the false negative the related-work literature identifies as the real cost of aggressive triage [7], [5]. On this benchmark the already-low false-positive population (144 known traps, a 1.4% FPR at **baseline**) leaves little genuine noise for an LLM triage stage to remove: the reported 88–98 % false-positive reductions in the static-analysis literature [4], [5], [7] are measured against corpora with a substantially higher false-positive rate than the one this DAST pipeline already achieves through its heuristic confirmation stage (section II-C) before triage ever runs: this benchmark is close to the wrong instrument for the question the stage is meant to answer, and a null suppression rate here is evidence about the target, not a validation of the triage stage’s safety or effectiveness in general. We report it anyway, instead of omitting an inconvenient arm, because the alternative — silently dropping the evaluation — would be worse scientific practice than an honest negative result. Whether the stage suppresses real noise, and at what recall cost, on a target with a deliberately elevated false-positive rate is the open question the future work of section VII takes up.

VI. THREATS TO VALIDITY

A. Internal validity: JVM warm-up and isolation

OWASP Benchmark is a Java application on a servlet container. The JVM’s JIT compiler optimizes hot code paths after several invocations, so the first requests to an endpoint are systematically slower than later ones. This introduces two biases: (i) the time-based confirmation stage (section II-C) could interpret warm-up latency as a payload-induced delay, inflating **CONFIRMED** findings in the early phases of the scan; and (ii) the detection-cost metric is measured in request index, not time, so it is robust to warm-up, but comparing absolute latency across configurations is not.

Mitigations applied: baseline latency is resampled immediately before every timed test (not once at the start); the threshold $\tau = 4$ s is generous relative to the observed warm-up variance; and every configuration runs against the same freshly started container, with a single client and no concurrent traffic, so any residual warm-up bias is common to every arm and cancels out in the paired contrast. Resource isolation (dedicated container, local network, no other scanners) reduces run-to-run variance, but does not eliminate that introduced by the OS scheduler or JVM garbage collection. A replication with explicit warm-up (N discard requests per endpoint) remains future work.

B. External validity: restriction to GET vectors

The study is limited to 338 injection points in **GET** query parameters, each saturating at 20 requests. This corpus is too small to exercise the combinatorial explosion

motivating section I’s architecture; the ablation demonstrates the scheduling mechanics work correctly under a modest, saturating budget, not that they scale favorably against a payload space large enough to actually explode. `POST` vectors with an `application/x-www-form-urlencoded` or `JSON` body, headers, cookies, and path parameters are out of scope. Consequently, the recall and cost results do *not* generalize directly to the full attack surface of a real application: categories with higher incidence in `POST` bodies (e.g., SQL injection in authentication forms) are under-represented. In addition, OWASP Benchmark is a synthetic testbed; its trigger payloads and response patterns are more regular than those of production applications, which likely overestimates any scanner’s absolute recall [1]. The contribution that does hold under this restriction is the *relative* one: the effect of each pipeline stage on a homogeneous, controlled target set.

The cross-tool comparison in section IV-F is a single run of each tool, not repeated trials, and the two scanners’ default policies test different, non-identical rule sets by design; a single run cannot rule out run-to-run variance on either side, and the *unrestricted* precision comparison in particular conflates “did not report this category” with “reported it incorrectly” for whichever scanner’s default policy is narrower in scope. Readers should treat table IV as one honest data point, not a leaderboard result generalizing to other targets, policies, or tool versions.

C. Construct and conclusion validity

The oracle collapses type aliases onto common buckets; a detection with the “correct” type but the “wrong” subclass (e.g., *blind* versus *error-based*) counts as a hit. FPR is defined only over the 144 explicit traps; false positives against tuples that ground truth does not mention have no reference population and are not scored. Finally, the number of blocks per budget is bounded by $|P| = 194$, sufficient for the Friedman test but modest for the Nemenyi post-hoc; critical-difference diagrams are interpreted as indicative rather than conclusive.

The LLM-triage evaluation (section V) carries an additional threat: the backend model, prompt, and `ai_fp_threshold` are fixed at their defaults for a single provider; the null suppression rate reported there is a property of that specific configuration on this benchmark, not a general claim about LLM-based triage.

VII. CONCLUSION AND FUTURE WORK

We presented `WEBSecurityScanner`, an asynchronous DAST engine whose architecture explicitly separates payload scheduling and finding confirmation into decoupled stages, together with a reproducible evaluation methodology on OWASP Benchmark, an independent oracle, an RCBD design, and a corpus-hashing manifest.

The ablation study yields a nuanced result. Over the 338 `GET`-parameter targets, the full heuristic pipeline reaches

70.6% recall at 1.4% FPR, and *saturates* already at 20 requests per parameter. Of the four scheduling optimizations evaluated, only **confidence-based prioritization** produces a practically consequential change, and it is bounded to tight budgets: disabling it costs 3.1 p.p. recall and 5.5 p.p. precision at budget 20, and nothing from 50 onward. **Context interleaving**, **runtime confirmation**, and **live adaptive sorting** do not change recall, precision, or FPR on this benchmark; the Friedman test detects differences in request *order* ($p = 7.8 \times 10^{-217}$), but with negligible effect sizes ($|\delta| < 0.02$). Layering **LLM-based triage** on top of the baseline pipeline suppressed no findings at any of the four evaluated budgets under the default confidence threshold, at no measured wall-clock cost (section V) — consistent with the same underlying explanation as the scheduling ablation: OWASP Benchmark’s trigger signals are regular enough that a conservative triage model finds nothing it is confident enough to drop. The operational conclusion is that, on a benchmark with trigger signals as regular as OWASP Benchmark’s, the benefit of payload scheduling is one of *anticipation* — higher area under the detection curve when the budget is tight — rather than a higher detection *ceiling*, and that neither runtime confirmation nor LLM triage penalize precision because the raw indicators are already reliable in this scenario.

Future work.

- 1) Repeat the external baseline of section IV-F with multiple trials of OWASP ZAP (and add Burp Suite and/or Nuclei) to turn the current single-run comparison into a statistically tested one, and restrict each tool’s active-scan policy to a matched rule set for a stricter apples-to-apples reading.
- 2) Repeat the study on real-world applications and benchmarks with noisy signals, where runtime confirmation and LLM-based triage should show the false-positive-reduction value that was not observable here.
- 3) Extend the benchmark to `POST` vectors, headers, and cookies to characterize categories that are currently under-represented.
- 4) Widen the grid to budgets ≤ 10 with replicates, where the effect of prioritization is largest, to narrow the confidence intervals.
- 5) Add explicit JVM warm-up and repeat the absolute-latency analysis across configurations.
- 6) Sweep the LLM triage confidence threshold and backend model, and evaluate triage on a corpus with a deliberately elevated false-positive rate to characterize the suppression/false-negative trade-off this benchmark could not exercise.
- 7) Publish the complete set of manifests and JSONL traces as a verifiable research artifact.
- 8) Extend the engine into the autonomous red-team pipeline currently under development in this repository — a LangGraph orchestrator coordinating at-

tacker, adaptive-fuzzer, CTI-ingestion, and reporting agents on top of WEBSECURITYSCANNER’s scanning core — and evaluate it as its own experimental subject once it reaches a stable interface.

ARTIFACT AVAILABILITY

The tool’s source code, the ground-truth generator, the experiment orchestrator, the grading oracle, and the analysis scripts are distributed with the project repository. Every reported run ships its own reproducibility manifest (section II-E).

REFERENCES

- [1] A. Doupé, M. Cova, and G. Vigna, “Why Johnny can’t pentest: An analysis of black-box web vulnerability scanners,” in *Proc. 7th Int. Conf. on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, 2010, pp. 111–131.
- [2] J. Bau, E. Bursztein, D. Gupta, and J. Mitchell, “State of the art: Automated black-box web application vulnerability testing,” in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2010, pp. 332–345.
- [3] M. Parvez, P. Zavarisky, and N. Khoury, “Analysis of effectiveness of black-box web application scanners in detection of stored SQL injection and stored XSS vulnerabilities,” in *Proc. 10th Int. Conf. for Internet Technology and Secured Transactions (ICITST)*, 2015, pp. 186–191.
- [4] A. Munson, J. Gomez, and Á. A. Cárdenas, “With a little help from my (LLM) friends: Enhancing static analysis with LLMs to detect software vulnerabilities,” in *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, 2025.
- [5] M. R. Ameen, M. T. U. Alam, and A. Islam, “QASecClaw: A multi-agent LLM approach for false positive reduction in static application security testing,” 2026.
- [6] V. Agrawal and K. Ahi, “LLM-driven SAST-genius: A hybrid static analysis framework for comprehensive and actionable security,” 2025.
- [7] X. Du, J. Feng, Y. Zou, W. Xu, J. Ma, W. Zhang, S. Liu, X. Peng, and Y. Lou, “Reducing false positives in static bug detection with LLMs: An empirical study in industry,” 2026.
- [8] D. She, A. Shah, and S. Jana, “Effective seed scheduling for fuzzing with graph centrality analysis,” in *2022 IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 2194–2211.
- [9] U.-S. Potti, H.-T. Chen, and H.-M. Sun, “Security testing framework for web applications: Benchmarking ZAP V2.12.0 and V2.13.0 by OWASP as an example,” 2025.
- [10] OWASP Foundation, “OWASP benchmark project,” <https://owasp.org/www-project-benchmark/>, 2023, accessed 2026.
- [11] M. Friedman, “The use of ranks to avoid the assumption of normality implicit in the analysis of variance,” *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
- [12] P. B. Nemenyi, “Distribution-free multiple comparisons,” *PhD thesis, Princeton University*, 1963.
- [13] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: A practical and powerful approach to multiple testing,” *Journal of the Royal Statistical Society: Series B*, vol. 57, no. 1, pp. 289–300, 1995.
- [14] N. Cliff, “Dominance statistics: Ordinal analyses to answer ordinal questions,” *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.
- [15] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.